CSE225 Data Structures **Fall 2011** **Student Name:_____**
Borahan Tümer & Fatma Ergin **Quiz #1 - Answers** **Student Number:_____**

Q1. (3+3+4=10pts)
Discuss, at most in a single paragraph, using visual aids (i.e., figures with boxes, arrows and other symbols we used to represent the main memory) what the following statements at each part of the question execute?

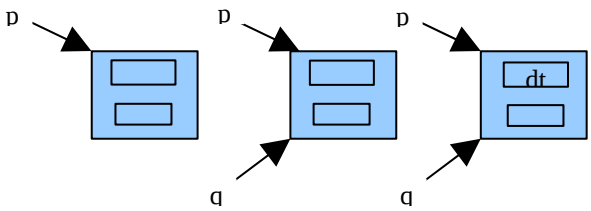
```
int k;
struct datatype {...};

struct nodetype {
   datatype data;
   struct nodetype *next;
};
typedef struct nodetype nodetype;
nodetype node, *p, *q;

datatype dt;
...
```

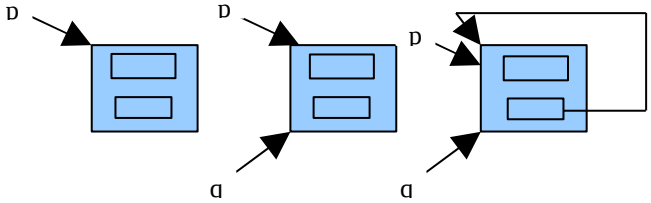
//question part a

```
scanf("%d",&k);
p=malloc(sizeof(nodetype));
q=p;
q->data=dt;
```



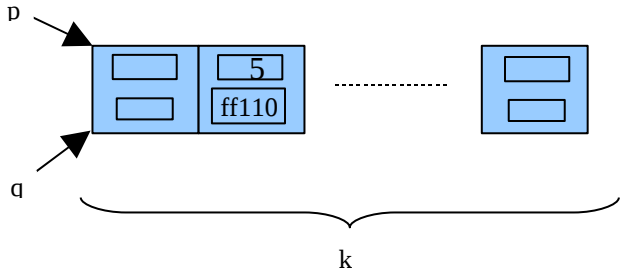
//question part b

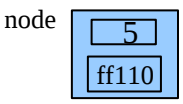
```
p=malloc(sizeof(nodetype));
q=p;
q->next=p;
```



//question part c

```
scanf("%d",&k);
p=calloc(k,sizeof(nodetype));
q=p;
node=*(q+1);
...
```





CSE225 Data Structures **Fall 2011** **Student Name:_____**
Borahan Tümer & Fatma Ergin **Quiz #1 - Answers** **Student Number:_____**

Q2. (7+7+6=20pts)

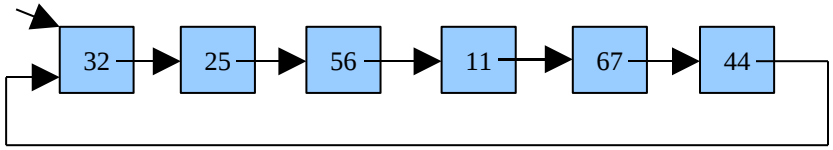
a) Why do we use an ampersand (&) in the scanf function's argument list?
"scanf" is a "call by reference" function. Functions can not return more than 1 value. The return value of "scanf" function is the number of items successfully read. So, we use "&" to store the result of this function on a regular variable.

b) We know that array names are pointer variables. Why, then, can we not use the name of an array to point to another array?
If we point an array with another, we would lose the address of the first array.

c) Why are arrays "static" and LLs dynamic structures?
When we define an array, the memory block is allocated in the first place, and it cannot be expanded any more. But, for linked lists, the memory is allocated during run time, and it can be expanded when needed.

Q3. (20+20 = 40pts)

a) Draw a circular linked list (CLL) from the following list of integers:
32 25 56 11 67 44



b) Write a small piece of code (including a while loop) that reads the above numbers from the file "inpf" and constructs the above CLL!

```
main(void){
        FILE *inpf;
        struct node {
           int x;
          struct node *next;
        };
        typedef struct node node;
        node *hdr, *p, *q;

        inpf = fopen("input.txt","r");
        p  = malloc(sizeof(node));
        hdr = p;

        fscanf(inpf,"%d",&p->x);
        p->next = hdr;
        while (!feof(inpf)) {
             q = malloc(sizeof(node));
             fscanf(inpf,"%lf",&q->x);
             p->next = q;
             q->next = hdr;
             p = q;
        }

        fclose(inpf);
}
```